New function

2025-05-20

Load gt

```
mad_bor_g1000_sum <-
  gt_load("../data/intermediate/rbind_data/mad_bor_g1000_sum_clean.gt",
    reattach_to = "../data/intermediate/rbind_data/mad_bor_g1000_sum_clean"
  )
mad_bor_g1000_sum <- mad_bor_g1000_sum %>% group_by(population)
```

Load fastmixture results

```
qmats <-
  readRDS("../data/intermediate/fastmixture/fastmixture_m.RData")
# check order
testthat::expect_equal(qmats$id, mad_bor_g1000_sum$id)
# add grouping variable
qmats$group <- mad_bor_g1000_sum$population
```

Add continental region

```
mad_bor_g1000_sum$Region <- "Other"
mad_bor_g1000_sum[mad_bor_g1000_sum$population %in% c("Borneo", "Sumba", "Timor"), "Region"] <- "Island
mad_bor_g1000_sum[mad_bor_g1000_sum$population %in% c("BEB", "PJL"), "Region"] <- "South Asia"
mad_bor_g1000_sum[mad_bor_g1000_sum$population %in% c("CHS", "KHV"), "Region"] <- "East Asia"
mad_bor_g1000_sum[mad_bor_g1000_sum$population %in% c("ESN", "GWD", "LWK", "MSL", "YRI"), "Region"] <-
mad_bor_g1000_sum[mad_bor_g1000_sum$population %in% c("Betsimisaraka", "Merina", "Diego", "Sakalava Tsi
```

```
qmats$continents <- mad_bor_g1000_sum$Region
```

New function

```
assigned_african <- list()

merina_df <- data.frame(mad_bor_g1000_sum[, c(1, 2)])
# create a dataframe of only merina individuals
merina_df <- merina_df %>% filter(population == "Merina")

for (this_k in 2:max(range(unique(qmats$k)))) { # for each k
  # list within a list - so we have $K containing runs 1 2 3
  assigned_african[[as.character(this_k)]] <- list()
  for (this_rep in 1:rle(qmats$k)$lengths[1]) { # for each run
```

```r
# get the q_matrix
k_run <- get_q_matrix(qmats, k = this_k, run = this_rep) %>% as.data.frame()
k_run$id <- qmats$id # add id
k_run$group <- qmats$group # add population

####  1. Create a factor column in the data.frame that assigns each sample
#         as either "African" or "non-African" based on the Population
#         column.
k_run$region <- qmats$continents # add the continent
k_run$region <- ifelse(k_run$region == "Africa", "African", "nonAfrican")

#### 2. Loop through the ancestry columns by grabbing the column references
#       as a list and do the following for each K to determine if it is an
#       African K or a non-African K:
k_evaluations <- list()

q_matrix_means <- k_run %>%
  group_by(region) %>%
  summarise(across(starts_with(".Q"), ~ mean(.x, na.rm = TRUE)))

# get the list of column names - each .Q value in this matrix
cols <- names(q_matrix_means)[grepl("^\\.Q", names(q_matrix_means))]

# Assign each column African or non-African based on the mean
for (i in cols) {
  if (q_matrix_means[[i]][q_matrix_means$region == "African"] > q_matrix_means[[i]][q_matrix_means$
    k_evaluations[[i]] <- "African"
  } else {
    k_evaluations[[i]] <- "nonAfrican"
  }
}

#### 3. Now that you've assigned each K column to "African" or "nonAfrican"
#       you can make a function that takes all the K columns as inputs and
#       returns a sum value if the column is "African".
sum_af <- function(k_run, k_evaluations) {
  k_run$sum_af <- 0
  for (i in names(k_evaluations)) {
    if (k_evaluations[[i]] == "African") {
      k_run$sum_af <- k_run$sum_af + k_run[[i]]
    }
  }
  return(k_run)
}
k_run <- sum_af(k_run, k_evaluations)

# And here we do the same for non-African
sum_as <- function(k_run, k_evaluations) {
  k_run$sum_as <- 0
  for (i in names(k_evaluations)) {
    if (k_evaluations[[i]] == "nonAfrican") {
      k_run$sum_as <- k_run$sum_as + k_run[[i]]
    }
```

```
      }
      return(k_run)
    }
    k_run <- sum_as(k_run, k_evaluations)

    # k_run now contains $sum_af and $sum_as - the sum of African and
    # non-African ancestry for each individual

    # create column name
    column <- paste0(paste0("k", this_k), paste0("_r", this_rep))

    #### 4. Assign African or non_African based on which sum is greater
    k_run <- k_run %>%
      mutate(
        !!column := ifelse(k_run$sum_af > k_run$sum_as, "African", "nonAfrican")
      ) %>%
      filter(group == "Merina") %>%
      select(column)

    #### 5. Bind to Merina data to return
    merina_df <- cbind(merina_df, k_run)
  }
}
```

```
## Warning: Using an external vector in selections was deprecated in tidyselect 1.1.0.
## i Please use `all_of()` or `any_of()` instead.
##   # Was:
##   data %>% select(column)
##
##   # Now:
##   data %>% select(all_of(column))
##
## See <https://tidyselect.r-lib.org/reference/faq-external-vector.html>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
merina_df[20:26, ]
```

```
##        id population      k2_r1      k2_r2      k2_r3      k3_r1      k3_r2
## 20 CH20RR     Merina nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican
## 21 CH21RR     Merina    African    African    African    African    African
## 22 CH22RR     Merina    African    African    African    African    African
## 23 CH23RR     Merina    African    African    African    African    African
## 24 CH24RR     Merina    African    African    African    African    African
## 25 CH25RR     Merina    African    African    African    African    African
## 26 CH26RR     Merina    African    African    African    African    African
##         k3_r3      k4_r1      k4_r2      k4_r3   k5_r1   k5_r2   k5_r3
## 20 nonAfrican nonAfrican nonAfrican nonAfrican African African African
## 21    African    African    African    African African African African
## 22    African    African    African    African African African African
## 23    African    African    African    African African African African
## 24    African    African    African    African African African African
## 25    African    African    African    African African African African
## 26    African    African    African    African African African African
```

```
##         k6_r1      k6_r2      k6_r3      k7_r1      k7_r2      k7_r3      k8_r1
## 20 nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican
## 21    African    African    African    African nonAfrican    African nonAfrican
## 22    African    African    African    African nonAfrican    African    African
## 23    African    African    African    African nonAfrican    African nonAfrican
## 24    African    African    African    African nonAfrican    African nonAfrican
## 25    African    African    African    African nonAfrican    African nonAfrican
## 26    African    African    African    African nonAfrican    African nonAfrican
##         k8_r2      k8_r3      k9_r1      k9_r2      k9_r3     k10_r1     k10_r2
## 20 nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican
## 21 nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican
## 22    African    African    African    African    African    African    African
## 23 nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican
## 24 nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican
## 25 nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican
## 26 nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican nonAfrican
##        k10_r3
## 20 nonAfrican
## 21 nonAfrican
## 22    African
## 23 nonAfrican
## 24 nonAfrican
## 25 nonAfrican
## 26 nonAfrican
```

Results show better consistency, this time up to K = 7. Last time the switch occured at K = 5.

```
assigned_african <- list()

merina_df_proportion <- data.frame(mad_bor_g1000_sum[, c(1, 2)])
# create a dataframe of only merina individuals
merina_df_proportion <- merina_df_proportion %>% filter(population == "Merina")

for (this_k in 2:max(range(unique(qmats$k)))) { # for each k
  # list within a list - so we have $K containing runs 1 2 3
  assigned_african[[as.character(this_k)]] <- list()
  for (this_rep in 1:rle(qmats$k)$lengths[1]) { # for each run
    # get the q_matrix
    k_run <- get_q_matrix(qmats, k = this_k, run = this_rep) %>% as.data.frame()
    k_run$id <- qmats$id # add id
    k_run$group <- qmats$group # add population

    ####  1. Create a factor column in the data.frame that assigns each sample
    #         as either "African" or "non-African" based on the Population
    #         column.
    k_run$region <- qmats$continents # add the continent
    k_run$region <- ifelse(k_run$region == "Africa", "African", "nonAfrican")

    #### 2. Loop through the ancestry columns by grabbing the column references
    #       as a list and do the following for each K to determine if it is an
    #       African K or a non-African K:
    k_evaluations <- list()

    q_matrix_means <- k_run %>%
      group_by(region) %>%
```

```
  summarise(across(starts_with(".Q"), ~ mean(.x, na.rm = TRUE)))

# get the list of column names - each .Q value in this matrix
cols <- names(q_matrix_means)[grepl("^\\.Q", names(q_matrix_means))]

# Assign each column African or non-African based on the mean
for (i in cols) {
  if (q_matrix_means[[i]][q_matrix_means$region == "African"] > q_matrix_means[[i]][q_matrix_means$
    k_evaluations[[i]] <- "African"
  } else {
    k_evaluations[[i]] <- "nonAfrican"
  }
}

#### 3. Now that you've assigned each K column to "African" or "nonAfrican"
#       you can make a function that takes all the K columns as inputs and
#       returns a sum value if the column is "African".
sum_af <- function(k_run, k_evaluations) {
  k_run$sum_af <- 0
  for (i in names(k_evaluations)) {
    if (k_evaluations[[i]] == "African") {
      k_run$sum_af <- k_run$sum_af + k_run[[i]]
    }
  }
  return(k_run)
}
k_run <- sum_af(k_run, k_evaluations)

# And here we do the same for non-African
sum_as <- function(k_run, k_evaluations) {
  k_run$sum_as <- 0
  for (i in names(k_evaluations)) {
    if (k_evaluations[[i]] == "nonAfrican") {
      k_run$sum_as <- k_run$sum_as + k_run[[i]]
    }
  }
  return(k_run)
}
k_run <- sum_as(k_run, k_evaluations)

# k_run now contains $sum_af and $sum_as - the sum of African and
# non-African ancestry for each individual

# create column name
column <- paste0(paste0("k", this_k), paste0("_r", this_rep))

#### SKIP  4. Assign African or non_African based on which sum is greater

k_run <- k_run %>%
  filter(group == "Merina") %>%
  mutate(
    !!column := sum_af
  ) %>%
```

```
      select(column)

    #### 5. Bind to Merina data to return
    merina_df_proportion <- cbind(merina_df_proportion, k_run)
  }
}
```

```
merina_df_proportion[20:26, ]
```

```
##        id population     k2_r1     k2_r2     k2_r3     k3_r1     k3_r2
## 20 CH20RR     Merina 0.4977184 0.4977244 0.4977424 0.4915287 0.4915341
## 21 CH21RR     Merina 0.6319143 0.6319195 0.6319318 0.6252233 0.6252337
## 22 CH22RR     Merina 0.6774599 0.6774640 0.6774748 0.6695765 0.6695847
## 23 CH23RR     Merina 0.6009898 0.6009951 0.6010085 0.5938839 0.5939009
## 24 CH24RR     Merina 0.6180061 0.6180109 0.6180240 0.6100007 0.6100138
## 25 CH25RR     Merina 0.6263919 0.6263966 0.6264094 0.6202125 0.6202274
## 26 CH26RR     Merina 0.5998931 0.5998988 0.5999121 0.5946759 0.5946775
##        k3_r3     k4_r1     k4_r2     k4_r3     k5_r1     k5_r2     k5_r3
## 20 0.4915279 0.4336855 0.4337288 0.4336973 0.5079426 0.5060308 0.5061551
## 21 0.6252268 0.5788662 0.5789035 0.5788751 0.6460327 0.6442806 0.6443994
## 22 0.6695814 0.6271765 0.6272147 0.6271716 0.6926317 0.6907439 0.6908885
## 23 0.5938888 0.5477545 0.5477094 0.5477545 0.6131439 0.6112619 0.6113933
## 24 0.6100063 0.5630241 0.5630540 0.5630331 0.6297961 0.6279422 0.6280776
## 25 0.6202205 0.5723642 0.5724018 0.5723735 0.6409045 0.6390904 0.6392269
## 26 0.5946817 0.5418410 0.5418868 0.5418506 0.6145245 0.6126239 0.6127569
##        k6_r1     k6_r2     k6_r3     k7_r1     k7_r2     k7_r3     k8_r1
## 20 0.4100265 0.4066798 0.4093749 0.4284102 0.2596080 0.4272310 0.2769552
## 21 0.5606031 0.5578659 0.5600711 0.5854653 0.4280659 0.5845047 0.4545156
## 22 0.6097610 0.6070318 0.6092112 0.6349728 0.4775030 0.6340360 0.5048009
## 23 0.5256216 0.5227629 0.5250627 0.5473349 0.3994318 0.5463272 0.4218000
## 24 0.5436182 0.5408702 0.5431018 0.5648687 0.4115980 0.5638466 0.4325442
## 25 0.5525494 0.5496510 0.5519968 0.5745187 0.4134882 0.5734733 0.4352551
## 26 0.5230990 0.5200236 0.5226426 0.5458537 0.3720355 0.5446892 0.3961444
##        k8_r2     k8_r3     k9_r1     k9_r2     k9_r3    k10_r1    k10_r2
## 20 0.2774990 0.2769473 0.2750454 0.2780772 0.2784567 0.2761537 0.2761089
## 21 0.4550741 0.4544768 0.4528431 0.4552312 0.4557071 0.4536315 0.4536268
## 22 0.5054936 0.5049910 0.5037966 0.5055417 0.5059601 0.5043259 0.5042259
## 23 0.4223550 0.4217586 0.4204152 0.4219669 0.4224310 0.4206408 0.4204699
## 24 0.4330405 0.4324168 0.4309160 0.4332495 0.4340644 0.4317609 0.4317398
## 25 0.4358284 0.4351938 0.4341460 0.4359282 0.4365765 0.4348844 0.4348618
## 26 0.3967696 0.3960852 0.3961280 0.3969942 0.3972342 0.3972882 0.3969862
##       k10_r3
## 20 0.2761691
## 21 0.4536734
## 22 0.5043204
## 23 0.4206194
## 24 0.4317733
## 25 0.4348953
## 26 0.3970924
```

```
long <- merina_df_proportion %>%
  pivot_longer(
    cols = -c(id, population),
    names_to = c("K_run"),
```

```
    values_to = "assigned"
  )


# Ensure your K_run is in the order it appears in the dataframe
long$K_run <- factor(long$K_run, levels = unique(long$K_run))

long %>% ggplot(aes(x = K_run, y = assigned, group = id, colour = id)) +
  geom_line() +
  geom_point() +
  labs(
    title = "Ancestry assignment",
    x = "K and run",
    y = "Ancestry assignment"
  ) +
  theme(axis.text.x = element_text(angle = 90)) +
  geom_hline(yintercept = 0.5)+
  ylim(0, 1)
```

